

ESPECIFICACIÓN DE REQUISITOS DE SOFTWARE

OpenFang Studio

Interfaz totalmente visual para crear, observar y operar agentes y subagentes sobre OpenFang.

Versión del documento	1.0
Fecha	4 de octubre de 2026
Organización	Aicore Agency
Estructura	Basada en ISO/IEC/IEEE 29148
Motor base	OpenFang v0.6.9 (binario oficial, checksum SHA-256 fijado)
Estado del producto	Fase 1 implementada; fases 2 a 5 especificadas y pendientes

Contenido

- 1. Introducción 4**
 - 1.1 Propósito 4
 - 1.2 Alcance 4
 - 1.3 Definiciones 4
 - 1.4 Convenciones 4
- 2. Descripción general 4**
 - 2.1 Perspectiva 4
 - 2.2 Usuarios 4
 - 2.3 Restricciones 5
 - 2.4 Supuestos 5
- 3. Hallazgos verificados sobre OpenFang v0.6.9 6**
 - 3.1 Hechos que sustentan el diseño 6
 - 3.2 Riesgo de seguridad: endpoints públicos 6
 - 3.3 Inconsistencias a vigilar 6
- 4. Requisitos funcionales 8**
 - 4.1 Mission Control 8
 - 4.2 Inspector de agente 8
 - 4.3 Actividad, trazas y replay 9
 - 4.4 Aprobaciones 9
 - 4.5 Constructor visual y asistente 9
 - 4.6 Operación: Hands, programación y canales 10
 - 4.7 Costos 10
 - 4.8 Seguridad y confianza 10
 - 4.9 Calidad, acceso y operación 11
- 5. Interfaces externas 12**
 - 5.1 API de OpenFang consumida por Studio (verificada en v0.6.9) 12
 - 5.2 API interna de Studio 12
 - 5.3 Eventos SSE de Studio 12
 - 5.4 Reglas de estado del nodo 12
- 6. Requisitos no funcionales 13**
- 7. Modelo de datos de Studio 13**
- 8. Casos de uso 14**
 - CU-01 Ver qué está pasando 14
 - CU-02 Aprobar una acción sensible 14
 - CU-03 Inspeccionar y probar un agente 14

CU-04 Crear un agente con asistente (fase 3) 14

CU-05 Depurar una ejecución (fases 2 y 5) 14

CU-06 Responder una alerta (fase 4) 14

9. Riesgos 14

10. Estado de verificación 15

Apéndice A. Variables de entorno 15

1. Introducción

1.1 Propósito

Este documento especifica los requisitos de **OpenFang Studio**, una aplicación web que permite ver cada agente y subagente como un nodo vivo, entender qué está haciendo y cómo está configurado, aprobar acciones sensibles y, en fases posteriores, crear y editar agentes con una interfaz gráfica. Es la referencia de diseño, desarrollo, pruebas y aceptación.

1.2 Alcance

Studio **no reemplaza** al motor: se apoya en OpenFang (runtime de agentes en Rust) y consume su API. Studio aporta la capa visual, la agregación de eventos, la operación (aprobaciones, pausa, costos) y, más adelante, el constructor y el asistente de creación. Se despliega con Docker en un VPS con Dokploy.

Fuera de alcance: modificar el núcleo de OpenFang (salvo huecos puntuales documentados), facturación, marketplace público y ajuste fino de modelos.

1.3 Definiciones

Término	Definición
OpenFang	Sistema operativo de agentes en Rust (un binario) con API REST, WebSocket y SSE, canales, Hands, MCP y auditoría.
Studio	Aplicación web de este documento (Next.js + React Flow) con un servidor intermedio (BFF).
BFF	Servidor intermedio de Studio: guarda la clave del motor, filtra rutas y normaliza eventos.
Hand	Paquete autónomo de OpenFang (HAND.toml, prompt y SKILL.md) que corre con horario y puertas de aprobación.
Cadena de auditoría	Registro encadenado por hash (SHA-256) de las acciones del motor; permite verificar integridad.
Mission Control	Pantalla principal: grafo vivo de agentes, actividad, aprobaciones y estado global.
Nodo	Representación visual de un agente; su color indica su estado.

1.4 Convenciones

Prioridad: **Alta**, **Media**, **Baja**. Estado: **Implementado** (existe y se probó), **Parcial**, **Pendiente**. Las metas numéricas de los requisitos no funcionales son propuestas ajustables.

2. Descripción general

2.1 Perspectiva

El navegador habla solo con Studio. El BFF de Studio habla con OpenFang por la red interna de Docker usando una clave Bearer que nunca llega al navegador. OpenFang no se publica en internet.

Componente	Responsabilidad	Tecnología
Studio (web)	Mapa de agentes, actividad, aprobaciones, inspector, chat de prueba	Next.js 14, React 18, React Flow
Studio (BFF)	Sesión, proxy con lista blanca, agregación de eventos (SSE)	Rutas de Next.js (Node)
OpenFang	Agentes, subagentes, herramientas, MCP, canales, Hands, presupuesto, auditoría	Rust, binario v0.6.9
Datos de Studio (fases 2+)	Vistas, layouts, comentarios, alertas, eventos históricos, usuarios	Postgres
Proxy / HTTPS	Dominio y certificados	Dokploy (Traefik)

2.2 Usuarios

Perfil	Necesidad
Operador	Ver qué hacen los agentes ahora, aprobar o rechazar acciones, pausar si algo falla
Creador	Crear y editar agentes con clics, probar y comparar versiones
Desarrollador	Revisar trazas, manifiestos, MCP y skills; integrar por API
Administrador	Gestionar claves de proveedores, límites de costo, accesos y actualizaciones del motor

2.3 Restricciones

- Despliegue con Docker Compose en un VPS propio con Dokploy; sin servicios gestionados obligatorios.
- Preferencia por APIs directas a los proveedores de modelos y componentes abiertos.
- OpenFang es pre-1.0: se fija versión y checksum y se verifica el contrato antes de actualizar.
- El puerto del motor nunca se publica (ver 3.2).
- Idioma de la interfaz: español.

2.4 Supuestos

- El operador dispone de al menos una clave de proveedor de modelos o de un modelo local.
- El motor expone, o se le puede añadir, el nivel de detalle de eventos que pide cada fase (ver RF de trazas).
- Los precios de modelos cambian: el costo mostrado depende de lo que reporte el motor.

3. Hallazgos verificados sobre OpenFang v0.6.9

Obtenidos leyendo el código fuente (commit de la versión 0.6.9) y ejecutando el binario oficial en un entorno de pruebas. Condicionan el diseño.

3.1 Hechos que sustentan el diseño

Hallazgo	Cómo se verificó	Consecuencia
Existe binario Linux x86_64 por versión, con archivo .sha256; el checksum descargado coincide	Descarga y sha256sum de la release v0.6.9	La imagen del motor no compila Rust: descarga el binario y verifica el hash
La imagen oficial de GHCR no está confirmada como pública (el compose del repo dice que no; consulta anónima dio 403)	Lectura de docker-compose.yml y prueba de acceso	No se depende de ella; se usa el binario fijado
GET /api/agents entrega id, nombre, estado, modelo, proveedor, is_inferencing, last_active, auth_status, identidad	Llamada real al API	El color de nodo 'pensando' sale de is_inferencing
GET /api/comms/topology entrega nodos y aristas ParentChild y Peer; las Peer salen del historial del bus (hasta 500 eventos)	Código y llamada real	Base del grafo; relaciones entre pares son aproximadas
La cadena de auditoría registra ToolInvoke, AgentMessage, AgentSpawn, AgentKill, CapabilityCheck, MemoryAccess, FileAccess, NetworkAccess, ShellExec, AuthAttempt, ConfigChange y WireConnect	Código (enum AuditAction)	Fuente de la actividad en vivo y del estado 'usando herramienta'
GET /api/audit/verify devuelve {valid, entries, tip_hash}	Llamada real	Indicador de integridad en la barra superior
El WebSocket por agente emite typing, phase, tool_start, tool_end, tool_result, text_delta y response	Lectura de ws.rs	Fuente de detalle fino de un agente seleccionado
Máximo 5 conexiones WebSocket por IP	Constante MAX_WS_PER_IP en ws.rs	No se puede abrir un WebSocket por agente desde el BFF: uno solo, para el agente seleccionado
Crear agentes: POST /api/agents con manifest_toml (201)	Llamada real con manifiestos oficiales	Base del constructor visual (fase 3)
Aprobaciones: GET /api/approvals y POST /api/approvals/{id}/approve reject	Código y llamada real	Bandeja de aprobaciones
Con clave configurada, los POST y varias lecturas exigen Bearer (401 sin clave, verificado desde IP no-loopback)	Prueba desde 192.0.2.2 contra el motor en 0.0.0.0	Studio envía siempre Bearer desde el servidor

3.2 Riesgo de seguridad: endpoints públicos

En v0.6.9, aunque se configure una clave, estos GET son **públicos** (respondieron 200 sin clave desde una IP no-loopback): /api/agents, /api/approvals, /api/budget, /api/config (con la clave enmascarada), /api/channels, /api/sessions, /api/hands, /api/status, /api/health, el panel web en / y el flujo /api/logs/stream (entradas de auditoría). **Por eso el motor jamás debe publicarse**; solo Studio, con su propia contraseña, queda accesible.

3.3 Inconsistencias a vigilar

- Las cifras del README y de la documentación del proyecto no coinciden entre sí (endpoints, proveedores, modelos y versiones citadas). En el código v0.6.9 se contaron 166 rutas registradas; no se validó independientemente ninguna cifra de rendimiento.

- El proyecto se declara pre-1.0 y avisa de cambios incompatibles entre versiones: se fija versión y se ejecuta `scripts/verify_contract.py` antes de actualizar.

4. Requisitos funcionales

Fase 1 está implementada y probada contra el motor real (servidor, proxy y flujo de eventos). La interfaz no se revisó en un navegador y el despliegue Docker/Dokploy no se probó.

4.1 Mission Control

ID	Requisito	Prioridad	Fase	Estado
RF-01	Mostrar cada agente como nodo con nombre, modelo y estado (en espera, pensando, usando herramienta, esperando aprobación, detenido, error).	Alta	F1	Implementado
RF-02	Mostrar relaciones padre-hijo y entre pares como aristas; resaltar con animación las de agentes activos.	Alta	F1	Implementado
RF-03	Actualizar el grafo en vivo (SSE) sin recargar; indicar si se pierde la conexión con el motor.	Alta	F1	Implementado
RF-04	Disposición automática por niveles y posiciones movibles que se recuerdan.	Media	F1	Parcial
RF-05	Barra de estado: conexión con el motor, gasto del día frente al límite, integridad de la auditoría, conteo de agentes activos y por aprobar.	Alta	F1	Implementado
RF-06	Aviso visible cuando un agente no tiene la clave de su proveedor.	Media	F1	Implementado
RF-07	Pausa global y por agente con confirmación.	Alta	F2	Parcial
RF-08	Agrupar y colapsar subárboles; filtros por Hand, canal o estado; vistas guardadas.	Media	F2	Pendiente
RF-09	Burbujas de mensajes sobre las aristas al pasar el cursor (últimos mensajes entre agentes).	Media	F2	Pendiente
RF-10	Posiciones y vistas guardadas en servidor (no solo en el navegador).	Media	F2	Pendiente

4.2 Inspector de agente

ID	Requisito	Prioridad	Fase	Estado
RF-11	Panel con modelo, proveedor, modo, respaldos, permisos de herramientas y red, bloqueos, skills asignadas, servidores MCP y sesión.	Alta	F1	Implementado
RF-12	Reiniciar y detener un agente; chat de prueba con costo y tokens.	Alta	F1	Implementado
RF-13	Resumen en lenguaje natural de qué hace el agente y qué puede o no puede hacer, generado desde su configuración.	Media	F2	Pendiente
RF-14	Pestañas de memoria (ver y olvidar datos), límites con barras de uso y conexiones con estado de salud.	Media	F2	Pendiente
RF-15	Historial de sesiones y ejecuciones; cada una abre su traza.	Alta	F2	Pendiente

4.3 Actividad, trazas y replay

ID	Requisito	Prioridad	Fase	Estado
RF-16	Feed de actividad en vivo desde la cadena de auditoría, con filtro por agente y marcas de resultado no exitoso.	Alta	F1	Implementado
RF-17	Detalle fino del agente seleccionado (herramienta iniciada y terminada, fases) mediante un único WebSocket.	Alta	F2	Pendiente
RF-18	Línea de tiempo por ejecución con carriles por agente, duración, costo y tokens.	Alta	F2	Pendiente
RF-19	Persistir eventos normalizados para consultar historial y reproducir.	Alta	F2	Pendiente
RF-20	Replay animado de una ejecución sobre el grafo.	Media	F5	Pendiente
RF-21	Bifurcar una ejecución desde un paso con otro modelo o instrucciones y comparar resultados.	Media	F5	Pendiente

4.4 Aprobaciones

ID	Requisito	Prioridad	Fase	Estado
RF-22	Bandeja con agente, herramienta y descripción; aprobar o rechazar con un clic; contador en la pestaña.	Alta	F1	Implementado
RF-23	Contexto ampliado (por qué lo pide, argumentos, historial reciente del agente).	Media	F2	Pendiente
RF-24	Notificaciones push y decisión desde el móvil (PWA).	Media	F4	Pendiente

4.5 Constructor visual y asistente

ID	Requisito	Prioridad	Fase	Estado
RF-25	Crear un agente desde plantilla o manifiesto (POST /api/agents) con formulario guiado.	Alta	F3	Pendiente
RF-26	Editar la configuración arrastrando y conectando nodos (herramientas, skills, MCP, subagentes, canales), con validación.	Alta	F3	Pendiente
RF-27	Modo avanzado: editar el manifiesto TOML en paralelo, sincronizado con el diagrama.	Media	F3	Pendiente
RF-28	Asistente: describir en lenguaje natural lo que se necesita y recibir un grafo propuesto como diff visual (nuevo, modificado) que se acepta con un clic.	Alta	F3	Pendiente
RF-29	Versionado de manifiestos y comparador de versiones lado a lado.	Media	F3	Pendiente
RF-30	Editor y probador de skills y Hands; importación desde repositorio.	Media	F3	Pendiente

4.6 Operación: Hands, programación y canales

ID	Requisito	Prioridad	Fase	Estado
RF-31	Lista de Hands con estado, activar o pausar, métricas y cola de aprobaciones propia.	Alta	F4	Pendiente
RF-32	Calendario de tareas programadas (cron y schedules) con último resultado y ejecución manual.	Media	F4	Pendiente
RF-33	Editor visual de workflows y su historial de ejecuciones.	Media	F4	Pendiente
RF-34	Pantalla de canales: estado, configuración, enrutamiento a agentes y QR de WhatsApp.	Alta	F4	Pendiente
RF-35	Catálogo de integraciones y servidores MCP con conexión guiada y estado de salud.	Media	F4	Pendiente
RF-36	Alertas por bucle, error repetido, agente caído o gasto cercano al límite (correo, Telegram o push).	Media	F4	Pendiente

4.7 Costos

ID	Requisito	Prioridad	Fase	Estado
RF-37	Gasto del día frente al límite y por agente (presupuestos del motor).	Alta	F2	Parcial
RF-38	Gasto por modelo y por día, proyección del mes y alertas al acercarse al límite.	Media	F2	Pendiente
RF-39	Editar límites de presupuesto por agente desde la interfaz.	Media	F4	Pendiente

4.8 Seguridad y confianza

ID	Requisito	Prioridad	Fase	Estado
RF-40	Contraseña de acceso con cookie firmada, límite de intentos y fallo cerrado si falta configuración.	Alta	F1	Implementado
RF-41	Lista blanca de rutas hacia el motor; la clave del motor solo vive en el servidor.	Alta	F1	Implementado
RF-42	Mapa de capacidades por agente con semáforo (qué puede tocar).	Media	F5	Pendiente
RF-43	Visor de la cadena de auditoría con filtros y verificación de integridad bajo demanda.	Media	F5	Pendiente
RF-44	Registro de auditoría propio de las acciones hechas desde Studio (quién aprobó, detuvo o cambió qué).	Alta	F5	Pendiente

4.9 Calidad, acceso y operación

ID	Requisito	Prioridad	Fase	Estado
RF-45	Evals por agente (casos con entrada y expectativas) ejecutables antes de activar una versión.	Media	F5	Pendiente
RF-46	Usuarios, espacios de trabajo y roles (propietario, administrador, editor, lector).	Media	F5	Pendiente
RF-47	Comentarios y notas sobre nodos y ejecuciones.	Baja	F5	Pendiente
RF-48	Exportar e importar un agente completo como paquete.	Baja	F5	Pendiente
RF-49	Migración guiada desde otros frameworks (usa la herramienta de migración del motor).	Baja	F5	Pendiente
RF-50	Despliegue reproducible con versión y checksum del motor fijados; script de verificación de contrato.	Alta	F0	Implementado
RF-51	Script para crear agentes de ejemplo de forma idempotente.	Media	F0	Implementado

5. Interfaces externas

5.1 API de OpenFang consumida por Studio (verificada en v0.6.9)

Endpoint	Uso en Studio	Auth en el motor
GET /api/agents · /api/agents/{id}	Lista y detalle de agentes	Lista pública; detalle según ruta
GET /api/agents/{id}/tools · skills · mcp_servers · session	Inspector	Bearer
POST /api/agents/{id}/message	Chat de prueba	Bearer
POST /api/agents/{id}/stop · restart	Control del agente	Bearer
GET /api/comms/topology	Nodos y aristas	Bearer
GET /api/approvals · POST /api/approvals/{id}/approve reject	Bandeja de aprobaciones	Lista pública; POST Bearer
GET /api/budget · /api/budget/agents	Gasto y límites	Públicos (GET)
GET /api/audit/recent · /api/audit/verify	Actividad e integridad	Bearer
POST /api/agents (manifest_toml)	Crear agentes (fase 3)	Bearer
GET /api/hands · /api/cron/jobs · /api/workflows · /api/channels	Operación (fase 4)	Públicos (GET)
WS /api/agents/{id}/ws	Eventos finos de un agente (fase 2)	Bearer o ?token=

5.2 API interna de Studio

Ruta	Función
POST /api/login	Valida la contraseña y entrega la cookie de sesión (5 intentos por minuto y por IP)
GET /api/events (SSE)	Flujo agregado: status, snapshot, activity, integrity
GET POST /api/off/[...path]	Proxy hacia el motor con lista blanca; rutas no permitidas devuelven 403

5.3 Eventos SSE de Studio

Evento	Contenido
status	{ok} — si el motor responde
snapshot	Agentes con estado normalizado, aristas, aprobaciones pendientes y presupuesto (solo cuando cambia)
activity	Entradas nuevas de auditoría con nombre de agente resuelto; la primera tanda se marca como backfill
integrity	{valid, entries}; se refresca cada 30 s

5.4 Reglas de estado del nodo

Estado	Regla
Detenido / error	state distinto de Running (error si el texto indica caída)
Esperando aprobación	Hay al menos una aprobación pendiente del agente
Pensando	is_inferencing verdadero en /api/agents
Usando herramienta	Hubo un ToolInvoke del agente en los últimos 4 s (aproximado por el sondeo de 1,5 s)
En espera	Ninguna de las anteriores

6. Requisitos no funcionales

Las metas son propuestas para validar con el uso real.

ID	Categoría	Requisito	Meta propuesta
RNF-01	Seguridad: exposición	El motor no se publica; solo Studio es accesible desde internet.	0 puertos del motor publicados
RNF-02	Seguridad: acceso	Acceso con contraseña; sin configuración Studio falla cerrado (503).	Sesión de 7 días, 5 intentos por minuto
RNF-03	Seguridad: secretos	La clave del motor y las de proveedores no llegan al navegador ni a las trazas.	Revisión de respuestas y código del cliente
RNF-04	Seguridad: superficie	El proxy solo permite las rutas que la interfaz necesita.	Lista blanca con pruebas de rutas prohibidas
RNF-05	Rendimiento	El estado de un nodo se refleja con poca demora.	Menos de 3 s en cambios de estado
RNF-06	Escalabilidad	El grafo sigue legible con decenas de agentes.	Agrupación y colapso; 50 agentes fluidos
RNF-07	Fiabilidad	Si el motor cae, Studio lo indica y se recupera solo.	Reconexión automática del SSE
RNF-08	Compatibilidad	Verificar el contrato antes de actualizar el motor.	verify_contract.py sin fallos
RNF-09	Portabilidad	Despliegue reproducible con Docker Compose en Dokploy.	Un despliegue desde cero en menos de 15 min
RNF-10	Usabilidad	Estados legibles de un vistazo; lenguaje claro; errores con acción sugerida.	Primer agente visible en menos de 2 min
RNF-11	Accesibilidad	Teclado, foco visible, contraste y reducción de movimiento.	WCAG AA como objetivo
RNF-12	Responsive	Utilizable en móvil, sobre todo aprobaciones.	Layout de una columna bajo 900 px
RNF-13	Mantenibilidad	Un único punto normaliza los eventos del motor; pruebas de contrato.	Cambios del motor se corrigen solo en el BFF
RNF-14	Observabilidad	Registro de acciones hechas desde Studio.	Auditoría propia (fase 5)
RNF-15	Respaldos	Copia de datos del motor (volumen /data) y de Postgres.	Copia diaria con restauración probada

7. Modelo de datos de Studio

Fase 1 no usa base de datos: las posiciones del grafo se guardan en el navegador y todo el estado viene del motor. A partir de la fase 2 se añade Postgres para lo que el motor no guarda.

Entidad	Contenido	Fase
events	Eventos normalizados (run, agente, padre, tipo, datos, costo, latencia) para historial y replay	2
views / layouts	Vistas guardadas y posiciones de nodos por usuario	2
alerts / alert_rules	Reglas y alertas disparadas	4
manifests / manifest_versions	Versiones de manifiestos creados desde Studio y su diff	3
users / workspaces / memberships	Cuentas, espacios y roles	5
comments	Notas sobre nodos y ejecuciones	5
studio_audit	Acciones realizadas desde Studio	5

Entidad	Contenido	Fase
eval_cases / eval_results	Casos de prueba y resultados por versión	5

8. Casos de uso

CU-01 Ver qué está pasando

Actor: Operador

1. Abre Studio e inicia sesión.
2. Ve el mapa con el color de cada agente y el feed de actividad.
3. Selecciona un agente y filtra el feed por él.

Alternativa: Motor caído: la barra lo indica y la vista se recupera al volver.

CU-02 Aprobar una acción sensible

Actor: Operador

1. Un agente pide usar una herramienta con aprobación; su nodo se pone ámbar.
2. Abre la pestaña Aprobaciones y lee qué pide.
3. Aprueba o rechaza; el agente continúa o recibe el rechazo.

Alternativa: Sin respuesta: depende del tiempo de espera del motor.

CU-03 Inspeccionar y probar un agente

Actor: Creador / desarrollador

1. Hace clic en un nodo.
2. Revisa modelo, permisos, skills y MCP.
3. Envía un mensaje de prueba y ve respuesta, costo y tokens.

Alternativa: Falta la clave del proveedor: el inspector lo marca y el chat muestra el error.

CU-04 Crear un agente con asistente (fase 3)

Actor: Creador

1. Describe lo que necesita en lenguaje natural.
2. Studio propone un grafo y muestra el diff visual.
3. Ajusta con clics, acepta y el agente se crea.

Alternativa: Manifiesto inválido: se muestran los errores por campo.

CU-05 Depurar una ejecución (fases 2 y 5)

Actor: Desarrollador

1. Abre una ejecución en el historial.
2. Revisa la línea de tiempo por agente y reproduce sobre el grafo.
3. Bifurca desde un paso con otro modelo y compara.

Alternativa: Eventos faltantes: se indica el hueco en la traza.

CU-06 Responder una alerta (fase 4)

Actor: Operador

1. Recibe una alerta de gasto o bucle en el móvil.
2. Abre Studio, ve el agente afectado y lo pausa.

Alternativa: Sin conexión: la alerta queda pendiente.

9. Riesgos

Riesgo	Impacto	Mitigación
Endpoints GET públicos del motor exponen agentes, aprobaciones y auditoría si se publica el puerto	Alto	No publicar el motor; solo Studio con contraseña (RNF-01)
Cambios incompatibles en el motor pre-1.0	Alto	Versión y checksum fijos, verify_contract.py, normalización en el BFF
Límite de 5 WebSocket por IP	Medio	Un WebSocket solo para el agente seleccionado; resto por auditoría
Estado 'usando herramienta' aproximado por sondeo	Medio	Eventos finos por WebSocket del seleccionado; si no basta, añadir un endpoint de eventos al motor
Aristas entre pares limitadas al historial del bus (500 eventos)	Bajo	Persistir eventos en Postgres (fase 2)
Grafo ilegible con muchos agentes	Medio	Agrupación, colapso, filtros, zoom semántico
Interfaz que controla agentes autónomos	Alto	Roles, confirmaciones, auditoría propia (fase 5)
Datos de marketing del proyecto inconsistentes	Bajo	Medir en el entorno propio antes de decidir

10. Estado de verificación

Elemento	Resultado
Binario v0.6.9: descarga, checksum y arranque	Verificado
Lectura del código del API, WebSocket, auditoría y autenticación	Verificado
Contrato (script de 10 comprobaciones contra el motor real)	Verificado
Autenticación desde IP no-loopback y lista de endpoints públicos	Verificado
Studio: compilación, login, middleware, proxy con lista blanca (403), SSE con snapshot, activity e integrity	Verificado
Creación de agentes por API y su aparición en el flujo	Verificado
Chat con un proveedor real de modelos	No probado
Build de las imágenes Docker y despliegue en Dokploy	No probado
Interfaz en navegador (render, interacción, móvil)	No probado

Apéndice A. Variables de entorno

Variable	Servicio	Uso
OPENFANG_API_KEY	motor y Studio	Clave Bearer entre Studio y el motor (obligatoria)
STUDIO_PASSWORD	Studio	Contraseña de acceso (obligatoria)
STUDIO_SECRET	Studio	Secreto para firmar la cookie de sesión (obligatoria)
ANTHROPIC_API_KEY, OPENAI_API_KEY, GEMINI_API_KEY, GROQ_API_KEY, OPENROUTER_API_KEY	motor	Claves de proveedores (al menos una)
OPENFANG_VERSION, OPENFANG_SHA256	construcción	Versión del motor y su checksum (se cambian juntos)
OPENFANG_URL	Studio	Dirección interna del motor (http://openfang:4200)